

UNIT 4: PUSHDOWN AUTOMATA AND TURING MACHINES

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Comprehensive Master Theory Guide	Course Code: PCC-303-ITT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 4.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Pushdown Automata (PDA) Fundamentals	Formal 7-tuple definition $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, Instantaneous Description (ID: (q, w, α)), Transition rules (push, pop, no-op).	High Priority (6-8 Marks Qs)
Modes of Acceptance & Equivalence	Acceptance by Final State $L(M)$ vs. Empty Stack $N(M)$, Equivalence algorithms $(L(M) \leftrightarrow N(M))$, Equivalence of CFG and PDA.	High Priority (8-10 Marks Qs)
Deterministic PDA (DPDA) & Parsing	Definition of DPDA, Strict subset relationship $(DCFL \subset CFL)$, Unambiguous parsing, LL(k) and LR(k) parser foundations.	High Priority (6-8 Marks Qs)
Turing Machine (TM) Fundamentals	Formal 7-tuple definition $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$, Infinite tape model, Read/Write head movements (L, R) , Instantaneous Description $(X_1 \dots X_{i-1} q X_i \dots X_k)$.	High Priority (8-10 Marks Qs)
Turing Machine Design & Computations	Designing TMs for languages $(a^n b^n, a^n b^n c^n, \{ww^R\})$, TMs as integer function computers (1's complement, addition, multiplication).	High Priority (8-10 Marks Qs)
Turing Machine Variants & Universal TM	Multi-Tape TM, Nondeterministic TM (NTM), Universal Turing Machine (UTM: stored-program model), Linear Bounded Automata (LBA).	High Priority (6-8 Marks Qs)

Pushdown Automata (PDA) Models

A Pushdown Automaton is a finite automaton equipped with an auxiliary Infinite Stack memory (Last-In, First-Out).

Formal 7-Tuple Definition of a Pushdown Automaton

DEFINITION:

A Non-Deterministic Pushdown Automaton (PDA) M is defined as a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where:

1. Q is a finite non-empty set of **control states**.
2. Σ is a finite non-empty set of **input symbols** (input alphabet).
3. Γ is a finite non-empty set of **stack symbols** (stack alphabet).
4. $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow P_f(Q \times \Gamma^*)$ is the **transition function**, mapping a state, optional input symbol, and top stack symbol to a finite set of next state and stack replacement string pairs.
5. $q_0 \in Q$ is the designated **initial state**.
6. $Z_0 \in \Gamma$ is the **start stack symbol**.
7. $F \subseteq Q$ is the set of **final (accepting) states**.

Instantaneous Description (ID) of a PDA

The complete configuration of a PDA at any step of computation is captured by its Instantaneous Description (ID).

DEFINITION:

An **Instantaneous Description (ID)** of a PDA is a triple:

$$(q, w, \gamma) \in Q \times \Sigma^* \times \Gamma^*$$

where:

- q is the current control state.
- w is the remaining unread input string.
- γ is the complete stack contents, written with the **top of stack at the leftmost symbol**.

Turnstile Transition Relation (I-): If $\delta(q, a, Z)$ contains (p, α) for $a \in \Sigma \cup \{\varepsilon\}$:

$$(q, aw, Z\beta) \mid - (p, w, \alpha\beta)$$

- **Pop Operation:** $\alpha = \varepsilon$ (removes Z from top of stack).
- **No-Op / Change Symbol:** $\alpha = Y$ (replaces top symbol Z with Y).
- **Push Operation:** $\alpha = YZ$ or $Y_1Y_2\dots Z$ (pushes new symbols onto stack above Z).

Modes of Acceptance: Final State vs. Empty Stack

IMPORTANT:

1. **Acceptance by Final State ($L(M)$):** The set of strings that drive the PDA from initial ID (q_0, w, Z_0) to a state in F , regardless of stack contents:

$$L(M) = \{w \in \Sigma^* : (q_0, w, Z_0) \mid -^* (p, \varepsilon, \gamma) \text{ for some } p \in F, \gamma \in \Gamma^*\}$$

2. **Acceptance by Empty Stack ($N(M)$):** The set of strings that empty the stack completely, regardless of the final state reached:

$$N(M) = \{w \in \Sigma^* : (q_0, w, Z_0) \mid -^* (p, \varepsilon, \varepsilon) \text{ for any } p \in Q\}$$

Equivalence Theorem: A language L is accepted by some PDA M_1 by final state if and only if L is accepted by some PDA M_2 by empty stack ($L(M_1) = N(M_2) = L$).

Deterministic Pushdown Automata (DPDA)

DEFINITION:

A PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is **Deterministic (DPDA)** if:

1. For any $q \in Q, a \in \Sigma, Z \in \Gamma$, the set $\delta(q, a, Z)$ contains at most one element.
2. If $\delta(q, \varepsilon, Z)$ is non-empty, then $\delta(q, a, Z)$ must be empty for all $a \in \Sigma$.

Structural Hierarchy: Deterministic Context-Free Languages (DCFL) form a **strict proper subset** of Context-Free Languages (DCFL $\subset$ CFL). For example, the language of even-length palindromes $L = \{ww^R : w \in \{a, b\}^*\}$ is a CFL but **not** a DCFL.

Source: Hopcroft et al., Ch 6, pp. 225–240 | Source: KLP Mishra, Ch 7, pp. 205–220

Equivalence of CFG and PDA

IMPORTANT:

Theorem of Equivalence: A language L is context-free ($L = L(G)$ for some CFG G) if and only if L is accepted by a Pushdown Automaton ($L = N(M)$ for some PDA M).

Conversion of CFG to PDA (Top-Down Parser Simulation)

Given CFG $G = (V, T, P, S)$, construct a 1-state PDA $M = (\{q\}, T, V \cup T, \delta, q, S, \emptyset)$ accepting by empty stack:

1. For the start state, place S on top of the stack ($Z_0 = S$).
2. For each non-terminal production $A \rightarrow \alpha \in P$, add rule:

$$\delta(q, \varepsilon, A) \text{ contains } (q, \alpha)$$

3. For each terminal symbol $a \in T$, add match rule:

$$\delta(q, a, a) = \{(q, \varepsilon)\}$$

Source: Hopcroft et al., Ch 6, pp. 241–250 | Source: KLP Mishra, Ch 7, pp. 221–228

Turing Machine (TM) Foundations

A Turing Machine is an abstract computing device possessing a finite control unit, a read/write head, and an **infinitely long two-way tape** divided into cells.

Formal 7-Tuple Definition of a Turing Machine

DEFINITION:

A Deterministic Turing Machine M is defined as a 7-tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where:

1. Q is a finite non-empty set of **control states**.
2. Σ is a finite non-empty set of **input symbols**.
3. Γ is a finite non-empty set of **tape symbols**, such that $\Sigma \subset \text{Strict } \Gamma$.
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the **transition function**, mapping current state and scanned tape symbol to a next state, a replacement symbol to write, and a head movement direction (Left L or Right R).
5. $q_0 \in Q$ is the designated **initial state**.
6. $B \in \Gamma - \Sigma$ is the **blank symbol**, filling all tape cells not containing input.
7. $F \subseteq Q$ is the set of **final (halting/accepting) states**.

Instantaneous Description (ID) of a Turing Machine

An ID of a Turing Machine represents the complete current configuration:

$$X_1 X_2 \dots X_{\{i-1\}} q X_i X_{\{i+1\}} \dots X_k$$

- The string $X_1 X_2 \dots X_k \in \Gamma^*$ represents non-blank tape contents.
- The control state $q \in Q$ is written immediately to the left of symbol X_i currently scanned by the read/write head.

Turnstile Transition Rules:

- If $\delta(q, X_i) = (p, Y, R)$, head moves Right:

$$X_1 \dots X_{\{i-1\}} q X_i X_{\{i+1\}} \dots X_k \mid - X_1 \dots X_{\{i-1\}} Y p X_{\{i+1\}} \dots X_k$$

- If $\delta(q, X_i) = (p, Y, L)$, head moves Left:

$$X_1 \dots X_{\{i-1\}} q X_i X_{\{i+1\}} \dots X_k \mid - X_1 \dots p X_{\{i-1\}} Y X_{\{i+1\}} \dots X_k$$

IMPORTANT:

Language Accepted by a Turing Machine:

$$L(M) = \{w \in \Sigma^* : q_0 w \mid -^* \alpha_1 p \alpha_2 \text{ for some } p \in F, \alpha_1, \alpha_2 \in \Gamma^*\}$$

A string w is accepted if M eventually enters an accepting state $p \in F$ and halts.

Source: Hopcroft et al., Ch 8, pp. 319–335 | Source: KLP Mishra, Ch 8, pp. 261–280

Turing Machine Design & Integer Computations

Turing Machines function as language acceptors and integer function computers.

INTUITION:**Turing Machine Design Logic for $a^n b^n c^n$:**

1. Scan left-to-right: Replace first unmarked a with X , search right for first unmarked b and replace with Y , search right for first unmarked c and replace with Z .
2. Move head left past X, Y, Z back to first unmarked a .
3. Repeat match cycle until all a, b, c symbols are converted to X, Y, Z .
4. Verify no remaining extra symbols exist. Enter final accepting state q_f .

Source: KLP Mishra, Ch 8, pp. 281–295 | Source: Hopcroft et al., Ch 8, pp. 336–342

Turing Machine Variants and Universal TM

1. Multi-Tape Turing Machine

Has multiple storage tapes, each with its own read/write head. A Multi-Tape TM with k tapes is **equivalent** in computational power to a standard single-tape TM (can be simulated with quadratic time slowdown $O(n^2)$).

2. Non-Deterministic Turing Machine (NTM)

The transition function maps to a finite set of next moves:

$$\delta : Q \times \Gamma \rightarrow P_f(Q \times \Gamma \times \{L, R\})$$

Every NTM is **equivalent** in computational recognition power to a Deterministic TM (simulated via breadth-first search of computation tree).

3. Universal Turing Machine (UTM)

A single static Turing Machine U that takes as input a string encoding of another TM M and an input w ($\langle M, w \rangle$), and simulates the execution of M on w . The UTM is the theoretical blueprint for modern general-purpose stored-program computers.

4. Linear Bounded Automata (LBA)

A non-deterministic Turing Machine whose tape head is restricted to move only within the bounded portion of tape occupied by initial input w . LBAs recognize Context-Sensitive Languages (Type 1).

Source: Hopcroft et al., Ch 8, pp. 343–360 | Source: KLP Mishra, Ch 8, pp. 296–310